

INTELLIGENT ADAPTIVE INSURANCE SUBMISSION AND UNDERWRITING ASSURANCE SYSTEM

A PROJECT REPORT

Submitted by

<STUDENT NAME 1> (<REG. NO.>)

<STUDENT NAME 2> (<REG. NO.>)

*in partial fulfillment of the requirement
for the award of the degree*

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY

(Autonomous)

Affiliated to Anna University, Chennai



SEPTEMBER - 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

- IM1.** To create next generation leaders by effective teaching learning methodologies and in still Scientifics park in them to meet the global challenges.
- IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.
- IM3.** To inculcate human values and ethical principles to cater the societal needs.
- IM4.** To contribute towards the research ecosystem by providing a suitable,

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science

applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and

novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI
(Autonomous)
Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

Certified that this project report titled “**INTELLIGENT ADAPTIVE INSURANCE SUBMISSION AND UNDERWRITING ASSURANCE SYSTEM**” is the bonafide work of **<STUDENT NAME 1> (<REG. NO.>), <STUDENT NAME 2> (<REG. NO.>)** who carried out the project (**Phase - I**) under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. S. Pavithra M.E., Ph. D

HEAD OF THE DEPARTMENT

Professor and Head,
Department of Computer Science and
Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE

Ms.S.Senthamizhselvi M.Tech

SUPERVISOR

Assistant Professor
Department of Computer Science and
Engineering
Chennai Institute of Technology
Chennai - 69

Submitted for the viva-voce examination held on

Internal Examiner

External Examiner

DECLARATION

We jointly declare that the project (**Phase - I**) report on **“INTELLIGENT ADAPTIVE INSURANCE SUBMISSION AND UNDERWRITING ASSURANCE SYSTEM”** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **“ANNA UNIVERSITY CHENNAI”** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This project report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.

Signature

Place: Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E, Ph.D.,** for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.,** for his effective leadership, encouragement and guidance in the project.

We would like to extend my thanks to our Year Coordinator **<NAME>** Project Co-ordinator **<NAME>** and Project Incharge **<NAME>** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **<NAME>** and **<NAME>** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

We wish to acknowledge the help received from various Departments and various individuals during the preparation and editing stages of the manuscript.

<STUDENT NAME>
<STUDENT NAME>

<Reg No. 123456>
<Reg No. 123456>

ABSTRACT

Commercial insurance submissions require different information, documents, validations and underwriting checks depending on the insurance product and on the characteristics of the applicant and the risk. Static forms and fixed document checklists cannot express this variability: they ask unnecessary questions, miss conditional requirements and, above all, confirm only that a document was uploaded, not whether the information it contains agrees with the rest of the submission.

This project designs an Intelligent Adaptive Insurance Submission and Underwriting Assurance System, a full-stack prototype modelled on a Guidewire PolicyCenter-oriented workflow. It determines the questions and documents a submission requires, extracts information with page-level provenance, normalizes it into canonical evidence and continuously checks the evolving submission for missing, conflicting, expired or insufficient evidence. Facts found in documents can trigger further requirements, and every requirement and issue carries a traceable reason.

The project does not claim novelty for AI document extraction or dynamic questionnaires. Its contribution is an evidence-assurance layer combining adaptive requirement determination, deterministic cross-document consistency checking with explicit tolerances and explainable requirement reasoning, presented through an underwriter dashboard with review actions and an audit trail. A large language model is used only for extraction and wording; decisions remain rule-based and reproducible. The stack is React with TypeScript, FastAPI and PostgreSQL.

Keywords: Commercial Insurance Underwriting, Guidewire PolicyCenter, Cross-Document Consistency, Evidence Assurance, Explainable Requirements, Intelligent Document Processing, Rule-Based Systems

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	vii
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF SYMBOLS AND ABBREVIATIONS	xii
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 MOTIVATION	2
	1.4 OBJECTIVES OF THE STUDY	3
	1.5 SCOPE OF THE PROJECT	3
	1.6 CONTRIBUTION AND NOVELTY	3
	1.7 ORGANIZATION OF THE REPORT	4
2	LITERATURE REVIEW	5
	2.1 COMMERCIAL INSURANCE SUBMISSION AND UNDERWRITING	5
	2.2 ADAPTIVE QUESTIONNAIRES AND RULE-BASED REQUIREMENTS	5
	2.3 INTELLIGENT DOCUMENT PROCESSING AND INFORMATION EXTRACTION	6
	2.4 ENTITY RESOLUTION, DATA FUSION AND CONFLICT DETECTION	6
	2.5 EXPLAINABILITY, PROVENANCE AND AUDITABILITY	6
	2.6 SUMMARY AND RESEARCH GAP	7
3	SYSTEM ANALYSIS AND DESIGN	8
	3.1 EXISTING SYSTEM	8
	3.2 PROPOSED SYSTEM	8
	3.3 FEASIBILITY STUDY	9
	3.4 REQUIREMENT ANALYSIS	9
	3.5 SYSTEM ARCHITECTURE	10
	3.6 PROCESS FLOW AND USE CASES	11
	3.7 SUBMISSION LIFE CYCLE	13
	3.8 DATABASE DESIGN	14
	3.9 INTERFACE AND SECURITY DESIGN	14
	3.10 USER INTERFACE DESIGN	15

4	METHODOLOGY	17
4.1	DEVELOPMENT APPROACH	17
4.2	CANONICAL EVIDENCE MODEL	17
4.3	REQUIREMENT ENGINE	18
4.4	DOCUMENT PROCESSING AND GROUNDED EXTRACTION	20
4.5	EVIDENCE NORMALIZATION	20
4.6	CROSS-DOCUMENT CONSISTENCY ENGINE	21
4.7	REQUIREMENT SUFFICIENCY AND EVIDENCE-TRIGGERED REQUIREMENTS	23
4.8	UNDERWRITING READINESS ASSESSMENT	23
4.9	EXPLANATION GENERATION	24
4.10	ROLE OF THE LLM AND GUARDRAILS	24
4.11	UNDERWRITER REVIEW WORKFLOW AND AUDIT TRAIL	24
4.12	EVALUATION PLAN	25
5	PROJECT WORK PLAN	26
5.1	WORK BREAKDOWN AND TIMELINE	26
5.2	TWO-DAY BUILD SPRINT PLAN	26
5.3	SCOPE CONTROL AND THE NOVELTY INVARIANT	27
5.4	RISK ANALYSIS AND MILESTONES	27
6	CONCLUSION AND FUTURE WORK	28
6.1	CONCLUSION	28
6.2	FUTURE ENHANCEMENTS	28
	REFERENCES	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Facts restated across typical commercial property submission documents	1
1.2	Positioning of project components	4
2.1	Summary of reviewed literature	7
2.2	Capability comparison of submission intake approaches	7
3.1	Feasibility study summary	9
3.2	Functional requirements	10
3.3	Principal REST endpoints	15
4.1	Canonical field dictionary (excerpt)	17
4.2	Requirements generated for the running example	19
4.3	Field evidence statuses	21
4.4	Consistency evaluation of the running example	22
4.5	Use of the LLM and deterministic controls	24
4.6	Underwriter actions and their effects	24
4.7	Evaluation metrics and targets	25
5.1	Two-day build sprint plan	26
5.2	MoSCoW prioritization under the novelty invariant	27
5.3	Risk register	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Checklist intake compared with evidence assurance for the same submission	2
3.1	Conceptual processing pipeline of the proposed system with its feedback loops	9
3.2	Layered system architecture	11
3.3	Submission life-cycle flowchart	12
3.4	Use case diagram (use cases realized by the evidence-assurance layer are highlighted)	13
3.5	Submission state transition diagram	13
3.6	Entity–relationship diagram in crow’s-foot notation (highlighted entities belong to the evidence-assurance layer)	14
3.7	Proposed wireframe: agent document requirements with reasons	16
3.8	Proposed wireframe: underwriter evidence and consistency view	16
4.1	Document processing and grounded extraction pipeline	20
5.1	Project timeline	26

LIST OF SYMBOLS AND ABBREVIATIONS

SYMBOLS

e	– Evidence item $\langle f, v, u, s, d, p, q, t, c \rangle$	–
E_f	– Set of active evidence items for canonical field f	–
f	– Canonical field key	–
v_i, v_j	– Normalized values being compared	unit of field
δ	– Relative difference between two values	%
τ_{ok}	– Agreement tolerance of a field	%
τ_{warn}	– Warning threshold of a field; beyond it a conflict is raised	%
θ	– Similarity threshold for text comparison	0 to 1
c	– Extraction confidence of an evidence item	0 to 1
C	– Completeness indicator	%
K	– Corroboration indicator	%
R	– Rule pack of an insurance product	–
h_k	– Hash of the k -th audit event	–
₹	– Indian rupee	–
M	– Million (10^6)	–

ABBREVIATIONS

ACORD	– Association for Cooperative Operations Research and Development
AI	– Artificial Intelligence
API	– Application Programming Interface
CP	– Commercial Property
DMN	– Decision Model and Notation
DPDP	– Digital Personal Data Protection
EIOPA	– European Insurance and Occupational Pensions Authority
GL	– General Liability
IDP	– Intelligent Document Processing
IRDAI	– Insurance Regulatory and Development Authority of India
JSON	– JavaScript Object Notation
JWT	– JSON Web Token
LLM	– Large Language Model
MoSCoW	– Must have, Should have, Could have, Will not have
NAIC	– National Association of Insurance Commissioners
OCR	– Optical Character Recognition
ORM	– Object–Relational Mapping

PDF	–	Portable Document Format
PIN	–	Postal Index Number
RBAC	–	Role-Based Access Control
REST	–	Representational State Transfer
SDS	–	Safety Data Sheet
SHA	–	Secure Hash Algorithm
SOV	–	Statement of Values
SPA	–	Single-Page Application
SUS	–	System Usability Scale
TIV	–	Total Insured Value
UI	–	User Interface
UW	–	Underwriter, Underwriting
XAI	–	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Commercial insurance protects businesses against property damage, liability, business interruption and related risks. A commercial submission must describe the business in detail: who the applicant is, what it does, where it operates, what it owns, how its premises are built and protected, and what losses it has suffered. An agent or broker prepares it, and an underwriter at the insurer decides whether, on what terms and at what price to accept the risk.

A submission is therefore a bundle of evidence: an application such as ACORD 125 with line sections such as ACORD 140, plus a statement of values (SOV), loss runs, financial statements, valuation reports and certificates [1]. Each document restates some of the same facts from a different source and time (Table 1.1); every fact reported twice is an opportunity for disagreement.

Table 1.1 Facts restated across typical commercial property submission documents

Fact	Application	SOV	Financials	Valuation	Loss runs	Fire certificate
Insured legal name	●	●	●	●	●	●
Risk location and address	●	●	○	●	○	●
Annual revenue	●	–	●	–	–	–
Building and contents values	●	●	○	●	–	–
Occupancy and construction	●	●	–	●	–	●
Prior losses	●	–	–	–	●	–
Hazardous materials	●	–	–	–	○	●

● *usually stated*; ○ *sometimes stated*; – *not applicable*.

In Guidewire PolicyCenter, a new-business submission is a job that progresses towards quoting, binding and issuance [2]. Two PolicyCenter constructs are central here: question sets for pre-qualification and underwriting questions, and underwriting issues, flags on a job that need approval within the reviewer’s authority [3]; referral reasons play a similar role for in-force policies [4]. The project adopts this vocabulary but implements it independently as a full-stack web application, not as a PolicyCenter configuration.

1.2 PROBLEM STATEMENT

Commercial submissions need different information, documents, validations and checks depending on the product, the applicant and the risk: a retail shop and a chemical factory may both seek property cover, yet need very different evidence. Static forms and fixed checklists cannot express this, and lead to unnecessary questions, incomplete submissions, inconsistent documents, repeated information requests and missed underwriting requirements.

Checklist intake answers the question “Has the document been uploaded?”. It does not answer the more important question “Does the information across the submission actually agree, and is it sufficient?” (Fig. 1.1). The problem addressed by this project is therefore:

How can an insurance submission be dynamically evaluated to determine what information and evidence is required, whether the submitted information is consistent and sufficient, and what still requires underwriting attention?

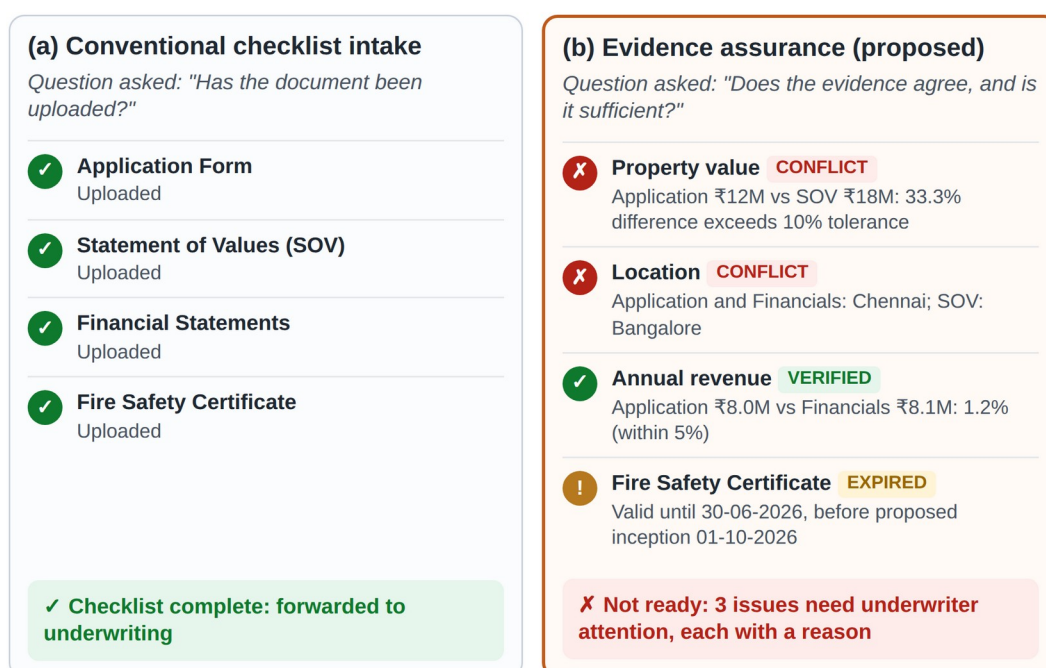


Figure 1.1 Checklist intake compared with evidence assurance for the same submission

1.3 MOTIVATION

Accenture and The Institutes found that underwriters spend around 40% of their time on administrative tasks and only about 30% on underwriting [5], much of it chasing documents and reconciling figures. Data quality and transparency are preconditions for AI

gains [6], and supervisors expect explainable, human-overseen, documented automation [7], [8]. Surfacing evidence gaps and conflicts early, with reasons and records, addresses both.

1.4 OBJECTIVES OF THE STUDY

1. Model a PolicyCenter-oriented submission workflow for two commercial lines.
2. Design a Requirement Engine that derives questions and documents from product, answers, risk and extracted evidence.
3. Design document extraction with source references, using a large language model (LLM) only where it helps.
4. Normalize all values to canonical fields, units, currencies and formats.
5. Design a deterministic Consistency Engine for conflicting, missing, expired and insufficient evidence.
6. Explain every requirement and issue by the rule and facts that produced it.
7. Provide an assurance dashboard, evidence view, underwriter actions and an audit trail.
8. Plan an evaluation on synthetic submissions with seeded inconsistencies.

1.5 SCOPE OF THE PROJECT

In scope: Agent, Underwriter and Administrator roles; Commercial Property and General Liability; application, SOV, financials, loss runs, valuation, certificate and hazardous-material documents; adaptive requirements, extraction, consistency, readiness, underwriter actions and audit trail, on synthetic data. **Out of scope:** rating, quoting and binding; live PolicyCenter integration; automated acceptance or declination; fraud scoring; external risk data; real personal data.

1.6 CONTRIBUTION AND NOVELTY

The project does not claim that AI document extraction [9] or dynamic questionnaires are new; both are established enabling components. The contribution is an evidence-assurance mechanism with two parts:

1. **Cross-document evidence consistency.** Not only “Did you upload the document?” but “Does the information agree?”. If the application states a property value of ₹12M and the SOV ₹18M, the system reports: *Conflict: property valuation differs between submitted sources (difference 33.3%, tolerance 10%); underwriter review required.*

2. **Explainable requirement reasoning.** Instead of “Upload Fire Safety Certificate”: *Required because the submission indicates an industrial property (Factory) with hazardous material storage.* Requirements triggered by facts found inside documents are explained the same way.

Novelty: an evidence-assurance layer that continuously evaluates the consistency and sufficiency of an evolving insurance submission and provides explainable reasons for additional underwriting requirements.

Table 1.2 positions each component; this combination, not “adaptive forms”, is a design invariant.

Table 1.2 Positioning of project components

Component	Status in practice and literature	Role in this project
Adaptive questionnaire	Established: question sets, rule engines [10]	Enabling
Document text extraction and OCR	Established: IDP products, [9], [11]	Enabling
Canonical evidence model with provenance	Known in data integration [12]	Enabling, built for assurance
Cross-document consistency with explicit tolerances	Rarely an explicit, explainable part of intake	Core contribution
Explainable and evidence-triggered requirements	Rule traces known from expert systems [13]; seldom shown to agents	Core contribution
Continuous readiness assessment of the evolving submission	Rarely an explicit part of intake	Core contribution

1.7 ORGANIZATION OF THE REPORT

Chapter 2 reviews literature; Chapter 3 presents analysis and design; Chapter 4 the methodology and evaluation plan; Chapter 5 the work plan; and Chapter 6 concludes.

CHAPTER 2

LITERATURE REVIEW

2.1 COMMERCIAL INSURANCE SUBMISSION AND UNDERWRITING

Underwriting selects, classifies and prices risks, and its quality depends on the completeness and accuracy of the submission. ACORD forms standardize the fields collected: the ACORD 125 master application and line sections such as ACORD 140, which records construction, occupancy, protection, year built, values and locations [1]. Standardized fields, however, do not guarantee that the values in different documents agree.

Guidewire PolicyCenter controls underwriting through underwriting issues: flags on a job, raised by configured rules, that can stop the job at a defined point until a user with sufficient authority approves them [3]; referral reasons capture similar concerns on in-force policies [4]. These concepts inform the UnderwritingIssue entity, the authority-based referral action and the blocking semantics of readiness in this project.

Aggour et al. automated long-term care and life underwriting at Genworth Financial with a fuzzy-logic rules engine encoding underwriter guidelines [14], showing that guidelines can be auditable rules at production scale; the system automates decisions on single applications rather than checking that a commercial submission's evidence is consistent. Eling et al. review AI along the insurance value chain from 91 papers and 22 industry studies and note a shift towards prediction and prevention, with open questions of data and transparency [6].

2.2 ADAPTIVE QUESTIONNAIRES AND RULE-BASED REQUIREMENTS

Conditional questioning is a mature technique, usually implemented with production rules or decision tables. Forgy's Rete algorithm made matching many rules against many facts efficient and underlies many business rules engines [15], and the OMG Decision Model and Notation (DMN 1.5) standardizes decision tables that business users and developers can share [10]. Two limitations matter here: intake rules decide which questions and documents appear but rarely show the reason, and they are evaluated over typed answers rather than over facts discovered in uploaded documents. The Requirement Engine of this project therefore uses DMN-inspired declarative rules, evaluated by forward chaining over both answers and extracted evidence, and records a trace for every requirement.

2.3 INTELLIGENT DOCUMENT PROCESSING AND INFORMATION EXTRACTION

Tesseract is a widely used open-source OCR engine [11]. LayoutLM models text with its page position [9], LayoutLMv3 unifies text and image masking [16], Donut removes the OCR stage [17], and FUNSD benchmarks noisy scanned forms [18]. LLMs allow extraction to be specified by instructions and a schema [19], [20], but may hallucinate content not in the source [21]; in evidence assurance this could create a false conflict or hide a real one, so each extracted value must carry its page and verbatim quote and is verified deterministically. Document processing optimizes accuracy per document; it does not check whether values from different documents agree.

2.4 ENTITY RESOLUTION, DATA FUSION AND CONFLICT DETECTION

Record linkage, from Fellegi and Sunter [22] to Christen [23], decides whether two values describe the same entity; the Levenshtein distance [24] and Jaro–Winkler comparator [25] tolerate variants such as “Pvt. Ltd.” and “Private Limited”. Data fusion merges representations of one object: Bleiholder and Naumann distinguish uncertainty from contradiction and classify strategies as conflict ignoring, avoiding and resolving [26], while truth discovery resolves conflicts automatically by estimating source reliability [27] and dependence [28]. Data-quality research treats consistency, completeness, accuracy and timeliness as separate dimensions [29], [30], and claim verification (FEVER) labels claims as supported, refuted or unverifiable [31], much as documents corroborate, contradict or leave unverified the values declared on an application.

Cross-document consistency in a submission is thus a small data-fusion problem with one crucial difference: automatic resolution is inappropriate, because choosing between conflicting values is an underwriting judgement. The design therefore detects and explains conflicts against explicit tolerances and may indicate the normally more authoritative source, but never overwrites a submitted value.

2.5 EXPLAINABILITY, PROVENANCE AND AUDITABILITY

MYCIN explained its conclusions by the rules it used [13] and XPLAIN added the justification behind rules [32]; for machine learning, LIME [33] and SHAP [34] approximate black boxes locally [35], [36]. Rudin argues that high-stakes decisions should use inherently interpretable models [37]; this project follows that view, so its explanations are exact, and it records both why- and where-provenance [12]. EIOPA [7] and the NAIC [8] stress

transparency, oversight and record keeping, and in India the IRDAI cyber-security guidelines [38] and the DPDP Act 2023 [39] govern security and personal data, motivating human oversight, an audit trail and synthetic data.

2.6 SUMMARY AND RESEARCH GAP

Table 2.1 summarizes the reviewed work and Table 2.2 compares approaches with the proposed system.

Table 2.1 Summary of reviewed literature

Work	Key contribution	Limitation for submission assurance
Aggour et al. [14]	Fuzzy rule engine encoding underwriting guidelines at scale	Automates decisions; no cross-document evidence checks
Guidewire [3], [4]	Underwriting issues, authority, referral, blocking points	Issues come from configured rules; document consistency is insurer-specific
LayoutLM [9], [16]; Donut [17]	Accurate form and layout understanding	Per-document extraction; no cross-document reasoning
LLM extraction [20]; hallucination [21]	Flexible schema-driven extraction; risk taxonomy	Outputs may be unfaithful; grounding needed
Data fusion [26], [27]	Conflict taxonomy; automatic truth discovery	Automatic resolution unsuitable where a human decides
Interpretability [35], [37]	Case for interpretable models in high-stakes settings	Model-centric; not tied to evidence workflows

Table 2.2 Capability comparison of submission intake approaches

Capability	Static form and checklist	Rule-based dynamic intake	Document extraction	Proposed system
Product-specific requirements	Partial	Yes	No	Yes
Requirements conditional on answers	No	Yes	No	Yes
Requirements triggered by document content	No	No	No	Yes
Structured extraction from documents	No	No	Yes	Yes
Cross-document consistency with tolerances	No	No	Partial	Yes
Explanation for every requirement and issue	No	Partial	No	Yes
Underwriter workflow and audit trail	Partial	Partial	No	Yes

No single approach combines requirement determination that reacts to document content, deterministic cross-document consistency with explicit tolerances, and an explanation for every requirement and issue within an underwriter workflow. This combination is the research gap addressed by the project.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 EXISTING SYSTEM

In checklist-driven intake, the agent fills a static application and attaches documents from a fixed checklist; an assistant confirms their presence and the underwriter reads them, compares figures manually and e-mails questions back. Every applicant gets the same questions; presence, not content or validity, is checked, so an expired certificate passes; inconsistencies surface late and depend on individual diligence; requests are ad hoc, repeated and unexplained; and nothing records what was checked and how it was resolved.

3.2 PROPOSED SYSTEM

The proposed Intelligent Adaptive Insurance Submission and Underwriting Assurance System replaces the static checklist with a continuously evaluated model of the submission (Fig. 3.1). The Requirement Engine determines the questions and documents that apply, each with a reason; uploaded documents are processed, and their fields are extracted with page-level provenance and normalized into canonical evidence; the Consistency Engine compares evidence across sources; requirements are re-evaluated for sufficiency; and the Underwriting Assurance component summarizes readiness and open issues for the underwriter.

Two feedback loops distinguish the design from a linear form. Facts discovered in documents flow back into the Requirement Engine and can create requirements, for example when an SOV lists an undeclared location; and underwriter requests for information become tracked requirements for the agent. Every change triggers deterministic re-evaluation, so the assurance view always reflects the current submission. The system flags and explains; underwriters decide.

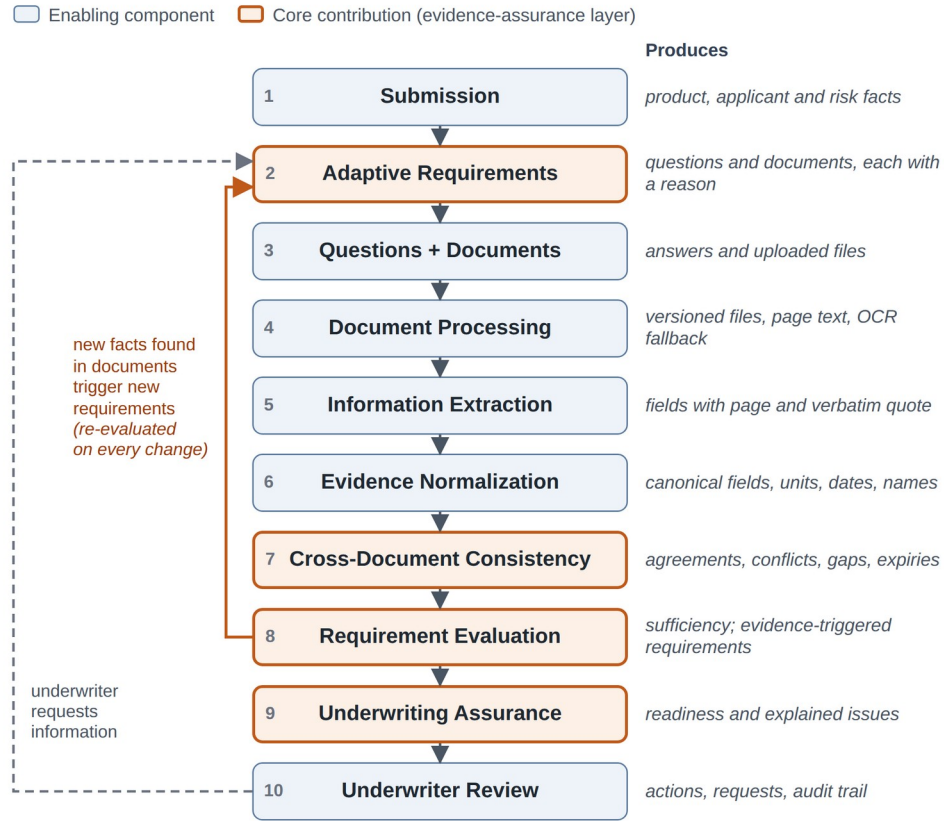


Figure 3.1 Conceptual processing pipeline of the proposed system with its feedback loops

3.3 FEASIBILITY STUDY

Table 3.1 Feasibility study summary

Dimension	Assessment	Basis
Technical	Feasible	Mature open-source stack (React, FastAPI, PostgreSQL); PDF text extraction and Tesseract OCR available; LLM APIs return schema-constrained JSON; engines are plain Python with no model training, so the system runs on a laptop.
Economic	Low cost	Open-source software; free hosting tiers for the front end (Vercel), API (Render or Railway) and PostgreSQL; LLM use is bounded because extraction runs once per document version and is cached by content hash.
Operational	Feasible	Mirrors existing agent and underwriter work (submissions, documents, issues, referral); explanations reduce back-and-forth.
Schedule	Feasible with scope control	Core scope sized for a two-day build sprint followed by evaluation; MoSCoW prioritization protects the core (Chapter 5).
Legal and ethical	Feasible with safeguards	Synthetic data only; no automated decisions about applicants; human oversight; data minimization consistent with the DPDP Act 2023 [39].

3.4 REQUIREMENT ANALYSIS

The system has three roles. The **Agent/Applicant** creates submissions, answers questions, uploads documents and responds to information requests, and sees only their own

submissions. The **Underwriter** reviews submissions, evidence and issues and takes actions within an authority limit. The **Administrator** maintains rule packs, tolerances and users. Table 3.2 lists the functional requirements.

Table 3.2 Functional requirements

ID	Requirement	Priority
FR-01	Authenticate users and enforce role- and ownership-based access with JWT bearer tokens.	Must
FR-02	Create a submission, select a product and enter initial applicant and risk information.	Must
FR-03	Show questions according to product, previous answers, risk characteristics and conditional rules.	Must
FR-04	Maintain a dynamic document checklist in which each requirement shows its status and reason.	Must
FR-05	Upload, replace and version documents and show their processing status.	Must
FR-06	Extract fields with source page and quote, and normalize them to canonical fields and units.	Must
FR-07	Compare evidence across sources with configurable tolerances; detect missing, conflicting, expired and low-confidence evidence.	Must
FR-08	Generate additional requirements when new information changes the submission.	Must
FR-09	Attach a traceable explanation to every requirement and issue.	Must
FR-10	Assess readiness before a submission is sent to underwriting.	Must
FR-11	Provide underwriter dashboard, detail and evidence views with review, request, accept, resolve, refer and move-forward actions.	Must
FR-12	Record all requirement changes, uploads, issues, actions and status changes in an audit trail.	Must
FR-13	Allow administrators to load versioned rule packs and tolerances.	Should

Non-functional requirements: **explainability**, every requirement and issue references its rule, rule-pack version, triggering facts and sources; **determinism**, the same state and rule-pack version always give identical results; **performance**, re-evaluation of a typical submission in under 2 s with asynchronous extraction; **security**, salted slow password hashes, role and ownership checks and validated uploads; **privacy**, synthetic data and minimal text sent to the LLM; **usability**, status never shown by colour alone; and **maintainability**, rules and tolerances kept as data with unit-tested engines.

3.5 SYSTEM ARCHITECTURE

The system follows a layered architecture (Fig. 3.2). A React and TypeScript single-page application [40] provides the two role-specific interfaces and calls a FastAPI application layer [41] over HTTPS; routers authenticate requests with JSON Web Tokens [42] and delegate to domain engines, which persist state in PostgreSQL through SQLAlchemy [43], store files in a versioned document store and read versioned rule packs.

External services are limited to an LLM API, used for schema-guided extraction, label mapping and optional wording, and local Tesseract OCR.

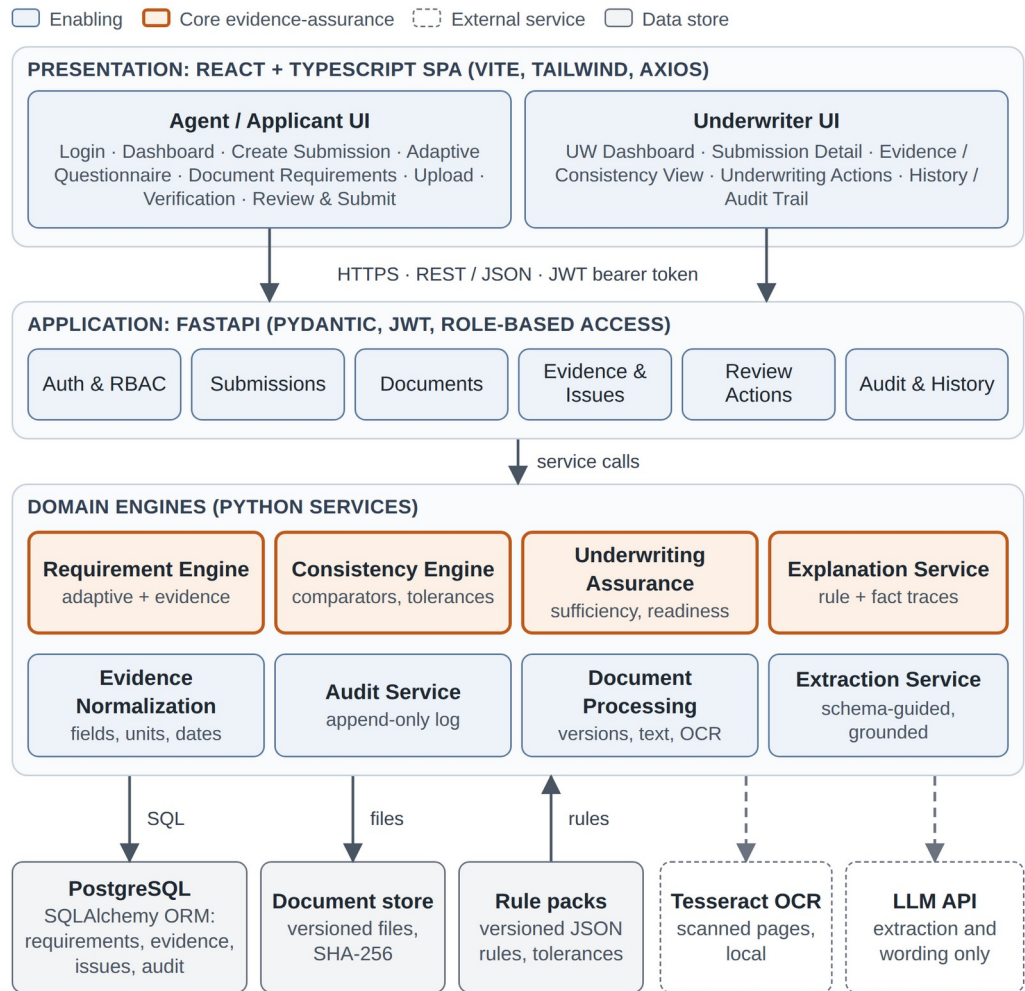


Figure 3.2 Layered system architecture

Each engine is a pure function of the submission state and the active rule-pack version, so it can be unit-tested without the user interface or the LLM and every result is reproducible for audit. Extraction runs as a background task; when it completes, the assurance loop re-evaluates the submission and the interface refreshes from the assurance endpoint.

3.6 PROCESS FLOW AND USE CASES

Fig. 3.3 shows the submission life cycle. After product selection, the Requirement Engine determines questions and documents; answers and uploads are processed and checked; new requirements loop back, and a submission that is not ready shows its gaps and reasons to the agent. Once ready, it goes to underwriting, where the underwriter requests information, refers the submission when an issue exceeds authority, or moves it forward to

quoting. Fig. 3.4 shows the corresponding use cases; the LLM service is a secondary actor used only in the included “Extract and verify evidence” use case.

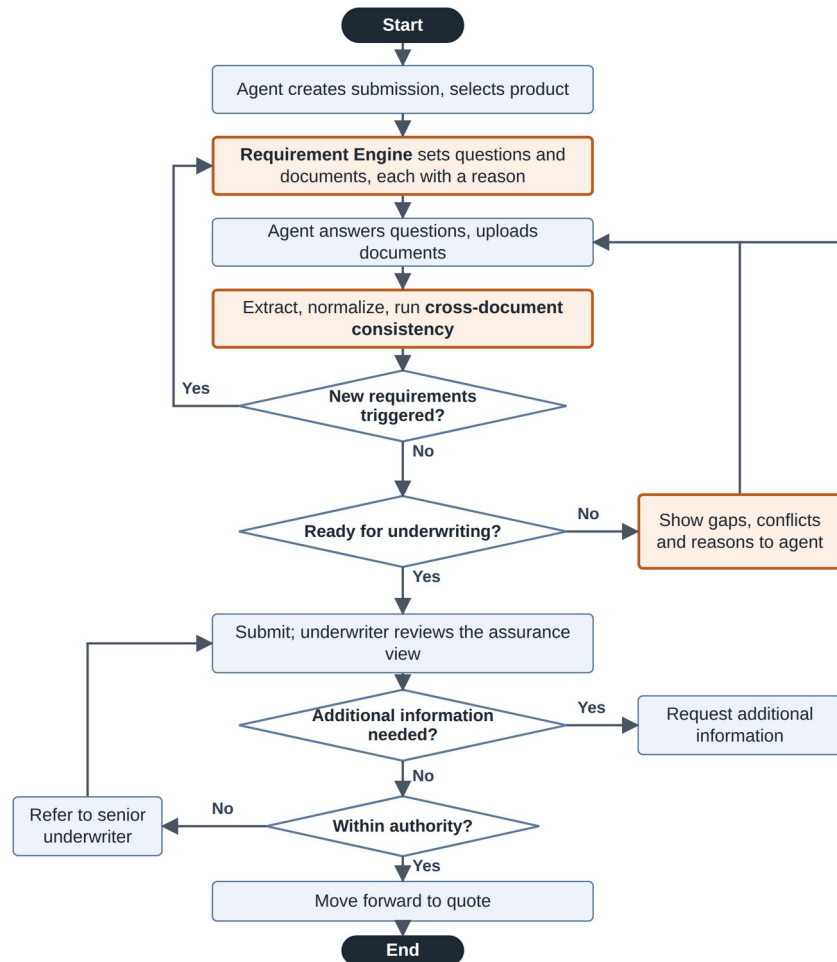


Figure 3.3 Submission life-cycle flowchart

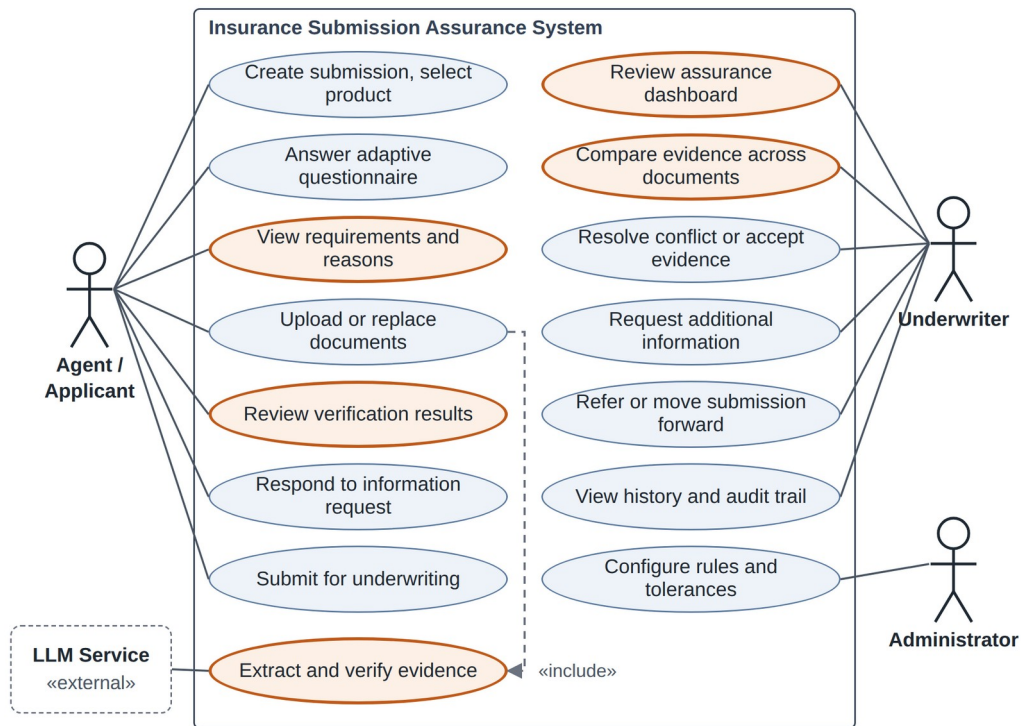


Figure 3.4 Use case diagram (use cases realized by the evidence-assurance layer are highlighted)

3.7 SUBMISSION LIFE CYCLE

Fig. 3.5 shows the submission states. The move from IN PROGRESS to READY is guarded by the readiness gates of Chapter 4 and reverses automatically when new evidence creates a requirement, conflict or expiry. After submission, the underwriter may request information, refer the submission, or move it forward to quoting.

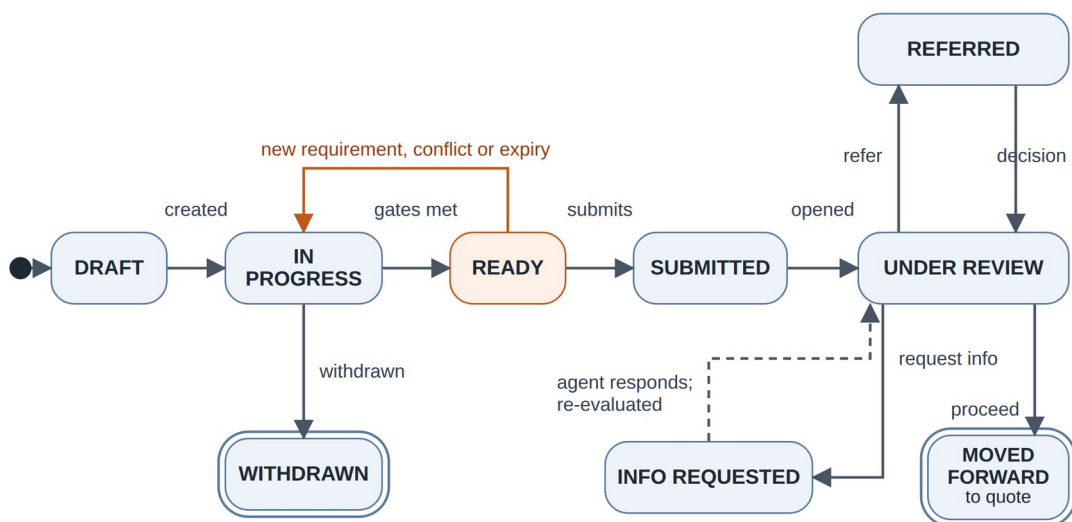


Figure 3.5 Submission state transition diagram

3.8 DATABASE DESIGN

Fig. 3.6 shows the entity–relationship model. Four decisions support assurance: evidence is first-class, with every ExtractedField row keeping the raw and normalized value, unit, page, quote and confidence; documents are versioned, and replacement supersedes rather than deletes evidence; requirements and issues store structured JSON explanations with the rule identifier and rule-pack version, keeping results reproducible after rules change; and AuditEvent is append-only and hash-chained.

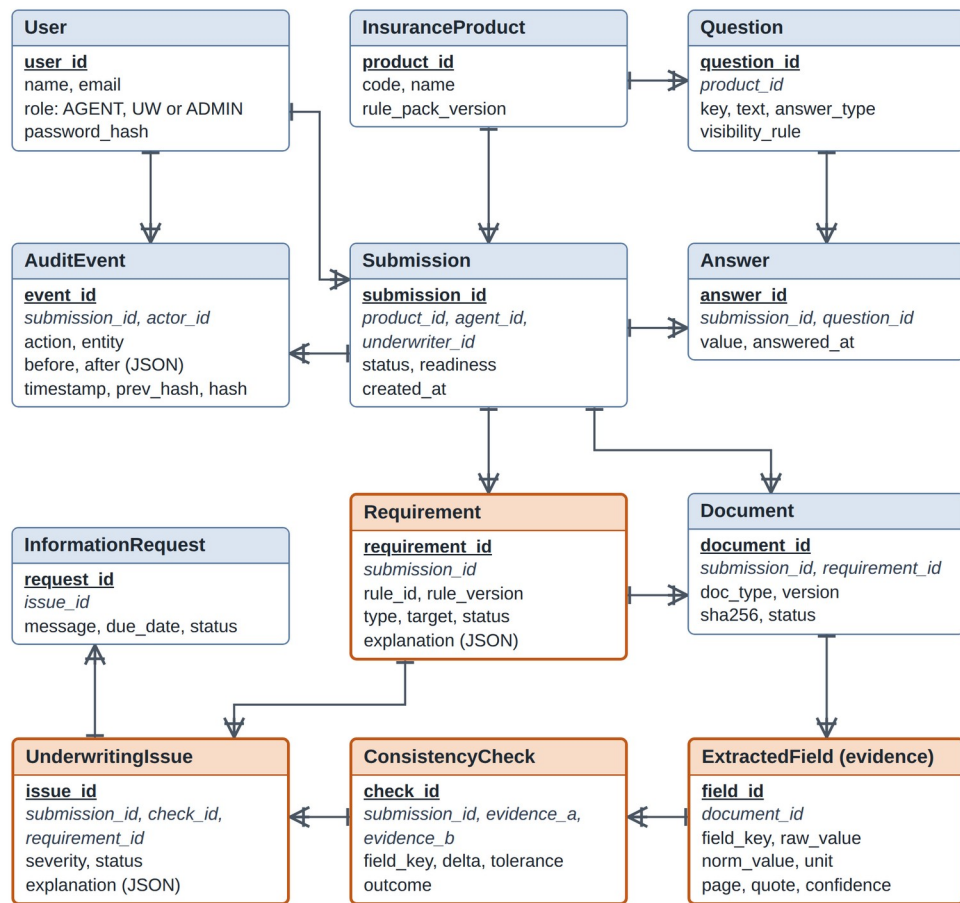


Figure 3.6 Entity–relationship diagram in crow’s-foot notation (highlighted entities belong to the evidence-assurance layer)

3.9 INTERFACE AND SECURITY DESIGN

The back end exposes a REST interface with JSON payloads (Table 3.3); every endpoint except login needs a bearer token and every write is audited. Security relies on short-lived JWTs [42], salted slow password hashes, role and ownership checks, validated and hashed uploads stored outside the web root, parameterized ORM queries and Pydantic validation against the OWASP Top 10 [44]. LLM calls send only necessary text and their

output is treated as untrusted; any use of real data would follow the DPDP Act 2023 [39] and IRDAI guidelines [38].

Table 3.3 Principal REST endpoints

Method	Endpoint	Role	Purpose
POST	/submissions	Agent	Create submission for a product
PUT	/submissions/{id}/answers	Agent	Save answers; re-evaluate
POST	/submissions/{id}/documents	Agent	Upload or replace a document
GET	/submissions/{id}/assurance	Agent, UW	Requirements, evidence, issues, readiness
POST	/issues/{id}/actions	UW	Review, accept, resolve, refer
POST	/submissions/{id}/info-requests	UW	Request additional information

3.10 USER INTERFACE DESIGN

The agent experience has eight pages: Login, Dashboard (submissions, status, pending actions), Create Submission (product and applicant), Adaptive Questionnaire, Document Requirements (dynamic checklist with status and reason), Document Upload (replace, extraction status), Submission Verification (verified fields, conflicts, gaps, warnings) and Review and Submit (readiness summary). The underwriter experience has five: Underwriter Dashboard (new, incomplete, conflicting and pending submissions), Submission Detail, Evidence/Consistency View, Underwriting Actions and History/Audit Trail. Figs. 3.7 and 3.8 are proposed wireframes of the two pages that carry the evidence-assurance layer; they are design mock-ups, not screenshots of a finished system.

/submissions/SUB-2026-0142/documents

Submission AssuranceAgent portal

SUB-2026-0142 · ABC Chemicals Pvt Ltd
Commercial Property · Factory, Chennai · Proposed inception 01-10-2026
NOT READY · 3 open issues

✓ Applicant
✓ Questions
Documents
Verification
Submit

Document requirements

6 required · 2 satisfied · 1 expired · 3 pending; each shows why it applies.

Requirement	Status	Why is this required?	
✓ Application Form	Satisfied	Base requirement CP-R01	View
✓ Loss History (5 years)	Satisfied	Factory occupancy CP-R05	View
! Fire Safety Certificate	Expired	Factory + hazardous material storage CP-R07; uploaded certificate expires 30-06-2026, before inception.	Replace
○ Valuation Report	Pending	Insured value ≥ ₹10M CP-R03; also settles the ₹12M vs ₹18M conflict.	Upload
○ Hazardous Material Docs	Pending	Hazardous material = Yes (Q12) CP-R08	Upload
○ Location Confirmation NEW	Pending	Triggered by evidence: SOV lists an undeclared Bangalore location CP-E04	Upload

Figure 3.7 Proposed wireframe: agent document requirements with reasons

/underwriting/SUB-2026-0142/evidence

Submission AssuranceUnderwriter workspace

SUB-2026-0142 · ABC Chemicals Pvt Ltd
Commercial Property · Submitted by agent · Readiness evaluated 2 minutes ago
NOT READY · 2 high · 1 medium

OverviewDocuments (6)**Evidence consistency**Issues (3)History

Field	Application	Financials	Statement of Values	Status
Insured name	ABC Chemicals Pvt Ltd	ABC Chemicals Private Limited	ABC Chemicals Pvt. Ltd.	✓ Verified names match after normalization
Annual revenue	₹8.0M	₹8.1M	—	✓ Verified Δ 1.2% ≤ 5% tolerance
Total insured value	₹12.0M	—	₹18.0M	✗ Conflict · High Δ 33.3% > 10% tolerance
Location (city)	Chennai	Chennai	Bangalore	✗ Conflict · High city differs across sources
Hazardous material	Yes	—	—	! Single source · Medium corroboration requested (CP-R08)

ISSUE-003 · HIGH · Total insured value differs between submitted sources
 Application (page 2, Q14): ₹12,000,000 · Statement of Values (page 1, "Total Insured Value"): ₹18,000,000.
 Difference 33.3% exceeds the 10% tolerance of rule CC-TIV-01.
 Source guidance: a certified valuation report is the most authoritative source for this field (pending, requirement CP-R03).

Mark reviewedRequest informationAccept evidenceResolve conflictRefer

Figure 3.8 Proposed wireframe: underwriter evidence and consistency view

CHAPTER 4

METHODOLOGY

4.1 DEVELOPMENT APPROACH

Development is iterative and anchored by the frozen definition of Chapter 1 and two principles. **Deterministic first:** the requirement and consistency engines are built and tested with manually entered evidence before LLM extraction is connected. **Novelty invariant:** every feature must strengthen adaptive requirements, cross-document consistency or explainable assurance; others are deferred (Section 5.3).

4.2 CANONICAL EVIDENCE MODEL

All information in a submission, typed or extracted, is represented as evidence items. An evidence item is the tuple

$$e = \langle f, v, u, s, d, p, q, t, c \rangle \quad (4.1)$$

where f is the canonical field key, v the normalized value, u its unit or currency, s the source type (APPLICATION, SOV, FINANCIALS, VALUATION, LOSS_RUN, CERTIFICATE), d the document version (empty for answers), p the page, q the verbatim quote, t the as-of date and c the extraction confidence in $[0, 1]$. E_f denotes the active evidence for field f , taken from the latest version of each document. The canonical field dictionary (Table 4.1) declares each field’s type, sources, materiality and comparison rule.

Table 4.1 Canonical field dictionary (excerpt)

Field key	Type	Typical sources	Materiality	Comparison rule
applicant.legal_name	text	Application, financials, SOV, certificates	High	Normalized name; Jaro–Winkler ≥ 0.92
applicant.annual_revenue	money	Application, financials	Medium	$\delta \leq 5\%$ agree; $\leq 10\%$ minor variance
property.total_insured_value	money	Application, SOV, valuation	High	$\delta \leq 5\%$ agree; $\leq 10\%$ minor variance
location.city, location.pin	address	Application, SOV, certificates	High	Exact match after normalization
building.occupancy	category	Application, SOV, certificates	High	Equality after synonym mapping
building.construction	category	Application, SOV, valuation	Medium	Equality after synonym mapping
building.year_built	integer	Application, SOV, valuation	Low	Absolute difference ≤ 1 year

Field key	Type	Typical sources	Materiality	Comparison rule
hazmat.present	boolean	Application, safety data sheets, certificates	High	Equality; a safety data sheet implies Yes
losses.count_5y, losses.incurring_5y	integer, money	Application, loss runs	High	Counts equal; amounts $\delta \leq 5\%$
certificate.fire.valid_until	date	Fire safety certificate	High	Not earlier than policy inception

4.3 REQUIREMENT ENGINE

4.3.1 Rule Representation

Requirements come from declarative rules in versioned rule packs, one per product. A rule has an identifier, a condition over facts, an action that creates a requirement (a question, a document or corroborating evidence), a mandatory flag and an explanation template filled from the facts that satisfied the condition (Listing 4.1). Facts come from answers and from normalized evidence; the latter lets document content trigger requirements.

Listing 4.1 Rule CP-R07 in the rule-pack format

```
{
  "id": "CP-R07",
  "product": "COMMERCIAL_PROPERTY",
  "when": { "all": [
    { "fact": "building.occupancy", "in": ["FACTORY", "WAREHOUSE"] },
    { "fact": "hazmat.present", "eq": true } ] },
  "then": { "require_document": "FIRE_SAFETY_CERTIFICATE",
    "mandatory": true,
    "valid_on": "policy.inception_date" },
  "because": "Fire Safety Certificate required because the submission
    indicates an industrial property ({building.occupancy})
    with hazardous material storage."
}
```

4.3.2 Evaluation Algorithm

Rules are evaluated by forward chaining to a fixed point (Algorithm 4.1). Rules only add requirements and derived facts and each fires at most once, so the loop ends after at most $|R|$ iterations for rule pack R . The result is compared with the previous set: new requirements become PENDING, requirements whose conditions no longer hold become WAIVED with the reason recorded, and the difference is audited.

Algorithm 4.1 Requirement evaluation by forward chaining

```

EvaluateRequirements(S, R)
Input : submission S, rule pack R ordered by priority
Output: requirement set Q, each with an explanation trace
1  F <- Facts(S)           // answers and best-supported evidence
2  Q <- {} ; fired <- {}
3  repeat
4      changed <- false
5      for each rule r in R with r.id not in fired do
6          if r.when(F) then
7              q <- Requirement(r.then, trace = facts used by r.when)
8              q.explanation <- Fill(r.because, q.trace)
9              Q <- Q + {q} ; F <- F + Derived(r)
10             fired <- fired + {r.id} ; changed <- true
11 until not changed
12 D <- Diff(Q, PreviousRequirements(S))
13 apply D (new -> PENDING, removed -> WAIVED) ; Audit(D)
14 return Q

```

4.3.3 Worked Example

The running example is a Commercial Property submission for ABC Chemicals Pvt Ltd, a fictitious applicant: building type Factory, hazardous material Yes, declared total insured value ₹12M, location Chennai, proposed inception 01-10-2026. Table 4.2 shows the rules that fire. CP-R01 to CP-R08 fire on answers; CP-E04 fires later, when the extracted SOV reveals a Bangalore location absent from the application.

Table 4.2 Requirements generated for the running example

Rule	Condition satisfied	Requirement	Explanation shown to the agent
CP-R01	Product = Commercial Property	Application form; statement of values	Base requirements for Commercial Property
CP-R03	Total insured value $\geq$ ₹10M	Property valuation report, certified, at most 24 months old	Declared total insured value (₹12M) meets the ₹10M threshold
CP-R05	Occupancy = Factory	Detailed loss history (5 years)	Factory occupancy requires a detailed five-year loss history
CP-R07	Occupancy $\in$ {Factory, Warehouse} and hazardous material = Yes	Fire safety certificate valid at inception	Industrial property with hazardous material storage
CP-R08	Hazardous material = Yes	Hazardous material documentation (inventory, safety data sheets)	Hazardous material storage declared (question 12)
CP-E04	SOV location not among declared locations	Location schedule confirmation	SOV lists a Bangalore location not declared in the application

4.4 DOCUMENT PROCESSING AND GROUNDED EXTRACTION

In the document pipeline (Fig. 4.1), uploads are validated, hashed with SHA-256 and versioned; text comes from the PDF text layer, with Tesseract OCR [11] for sparse pages, and keyword signatures flag mislabelled uploads.

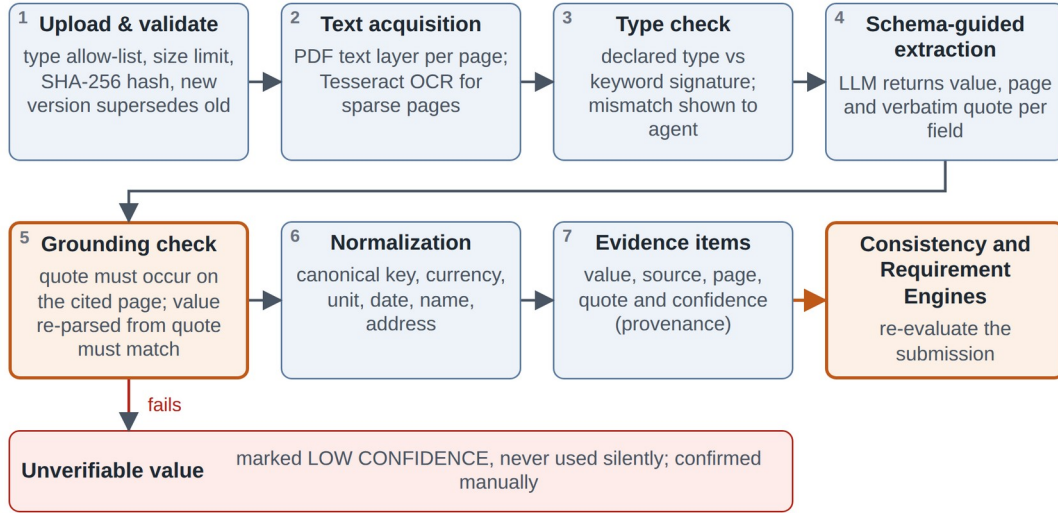


Figure 4.1 Document processing and grounded extraction pipeline

Extraction is schema-guided: for each field in the document type’s JSON Schema [45], the LLM returns the value, page and verbatim quote (for example “Total Insured Value: Rs. 1,80,00,000” on page 1 of the SOV). A deterministic grounding check then confirms that the quote occurs on the cited page and that the value parsed from it matches; failing values are marked LOW CONFIDENCE for manual confirmation, never used silently. Confidence is 0.95 for verified text-layer quotes, 0.85 for verified OCR quotes and at most 0.6 otherwise.

4.5 EVIDENCE NORMALIZATION

Normalization maps raw values to the canonical dictionary. Labels go through a synonym dictionary (“Total Insured Value”, “TIV” and “Sum Insured” all map to `property.total_insured_value`); the LLM may map an unmatched label only to an existing key. Money handles ₹, Rs., INR and USD, Indian digit grouping, lakh (10^5), crore (10^7) and suffixes such as Mn and Cr, so “Rs. 1.2 Crore” becomes 12,000,000 INR; dates are parsed day-first to ISO 8601 (“30/06/2026” becomes 2026-06-30); addresses yield PIN code and city with aliases such as Bengaluru and Bangalore; names are case-folded with legal suffixes unified (Private Limited and Pvt. Ltd. become PVT LTD); and categories map to controlled vocabularies (“manufacturing unit” becomes FACTORY).

4.6 CROSS-DOCUMENT CONSISTENCY ENGINE

4.6.1 Comparison Model

For every field f with evidence from at least two sources, each pair (e_i, e_j) is compared with the comparator declared for f . Numeric and monetary fields use the relative difference

$$\delta(v_i, v_j) = |v_i - v_j| / \max(|v_i|, |v_j|) \quad (4.2)$$

with two field-specific tolerances $\tau_{ok} < \tau_{warn}$: the pair AGREES when $\delta \leq \tau_{ok}$, shows a MINOR VARIANCE when $\tau_{ok} < \delta \leq \tau_{warn}$ and CONFLICTS when $\delta > \tau_{warn}$. Being symmetric and scale-free, δ applies equally to a ₹1M and a ₹100M property. Names use Jaro–Winkler similarity [25] with threshold $\theta = 0.92$, categories and booleans use equality after normalization, addresses compare PIN code and city, and dates use equality or an ordering constraint such as “valid on the inception date”.

4.6.2 Field Status

Pairwise outcomes, confidences and validity dates are aggregated into one status per field (Table 4.3).

Table 4.3 Field evidence statuses

Status	Condition	Consequence
VERIFIED	At least two independent sources agree and none conflicts	Shown as corroborated
MINOR VARIANCE	All pairs within τ_{warn} , at least one above τ_{ok}	Warning to the underwriter
CONFLICT	At least one pair beyond τ_{warn}	Issue whose severity depends on materiality
SINGLE SOURCE	Only one source provides the field	Corroboration requirement if the field is highly material
MISSING	Required field with no active evidence	Requirement stays PENDING
EXPIRED	Validity date earlier than the reference date, such as inception	Requirement EXPIRED; replacement requested
LOW CONFIDENCE	Evidence confidence below 0.7	Manual confirmation before use

4.6.3 Cross-Field Rules, Severity and Source Authority

Cross-field rules check relations between fields: SOV location values must sum to the total insured value, SOV locations must match those declared, a certificate’s address must match an insured location, and “no hazardous material” conflicts with an uploaded safety data sheet.

Severity combines materiality with the outcome: conflicts, expiries and missing evidence are High on high-materiality fields, Medium on medium ones and Low otherwise; minor variances, single-source and low-confidence values are one level lower. An authority order (valuation report, then SOV, then application for property values) lets an issue name the normally more authoritative source, but the engine never replaces a submitted value: it avoids rather than resolves conflicts [26], unlike truth discovery [27].

4.6.4 Consistency Evaluation Algorithm

Algorithm 4.2 summarizes the evaluation. With only a handful of evidence items per field, the pairwise cost $O(\sum_f |E_f|^2)$ is negligible, so the whole submission is re-evaluated after every change, keeping results simple and reproducible.

Algorithm 4.2 Cross-document consistency evaluation

```

EvaluateConsistency(S, D)
Input : submission S, canonical field dictionary D
Output: field statuses, consistency checks, issues
1 for each field f in D do
2   E <- ActiveEvidence(S, f)           // latest document versions only
3   flag and remove items with confidence < 0.7 or expired validity
4   if E is empty then status[f] <- MISSING if Required(S, f)
5   else if |Sources(E)| = 1 then status[f] <- SINGLE_SOURCE
6   else
7     for each pair (ei, ej) in E with source(ei) != source(ej) do
8       (delta, outcome) <- D[f].compare(ei.v, ej.v)
9       RecordCheck(f, ei, ej, delta, outcome, D[f].rule_id)
10    status[f] <- Aggregate(outcomes)
11    if IssueWorthy(f, status[f]) then
12      UpsertIssue(f, Severity(D[f].materiality, status[f]), Explain(f))
13 EvaluateCrossFieldRules(S)
14 auto-resolve issues whose condition no longer holds ; Audit(changes)
15 return status, checks, issues

```

4.6.5 Worked Example

Table 4.4 applies the engine to the running example; it is the comparison the underwriter sees in Fig. 3.8.

Table 4.4 Consistency evaluation of the running example

Field	Application	Financials	SOV	Computation	Outcome
Annual revenue	₹8.0M	₹8.1M	–	$\delta = 0.1 / 8.1 = 1.2\% \leq 5\%$	VERIFIED
Total insured value	₹12M	–	₹18M	$\delta = 6 / 18 = 33.3\% > 10\%$	CONFLICT, High
Location (city)	Chennai	Chennai	Bangalore	City mismatch	CONFLICT, High
Insured name	ABC Chemicals Pvt Ltd	ABC Chemicals Private Limited	ABC Chemicals Pvt. Ltd.	Equal after normalization	VERIFIED

The property value issue reads: *Conflict: property valuation differs between submitted sources. Application (page 2, question 14): ₹12,000,000. Statement of values (page 1, “Total Insured Value”): ₹18,000,000. Difference 33.3% exceeds the 10% tolerance of rule CC-TIV-01. A certified valuation report, the most authoritative source, is pending (CP-R03). Action: underwriter review required.* The location conflict also turns the Bangalore location into a fact that satisfies rule CP-E04, creating the location-confirmation requirement of Table 4.2: the closed loop between the two engines.

4.7 REQUIREMENT SUFFICIENCY AND EVIDENCE-TRIGGERED REQUIREMENTS

An upload alone does not satisfy a requirement. A requirement is SATISFIED only when (i) evidence of the required type is present, (ii) the document is of the declared type and valid at the reference date, (iii) its key fields were extracted with confidence of at least 0.7, and (iv) no High-severity conflict is open on those fields; otherwise it is PENDING, INSUFFICIENT or EXPIRED with the failing condition as its reason, or WAIVED by an underwriter with a justification. In the running example the fire safety certificate is valid only until 30-06-2026, before inception on 01-10-2026, so the requirement is EXPIRED with exactly that reason. Because consistency results and extracted values are themselves facts, the two engines run alternately until neither changes anything (two or three passes in practice, with an iteration bound), which makes the assurance continuous.

4.8 UNDERWRITING READINESS ASSESSMENT

Readiness is decided by deterministic gates rather than an opaque score:

- **NOT READY** if any mandatory requirement is PENDING, INSUFFICIENT or EXPIRED, or any High-severity issue is open;
- **READY WITH WARNINGS** if all mandatory requirements are satisfied but Medium or Low issues remain;
- **READY** if all mandatory requirements are satisfied and no issue above Low is open.

Two indicators, completeness C and corroboration K , show progress but do not decide readiness:

$$C = |\text{satisfied mandatory requirements}| / |\text{mandatory requirements}| \quad (4.3)$$

$$K = |\text{VERIFIED high-materiality fields}| / |\text{high-materiality fields with evidence}| \quad (4.4)$$

Submission to underwriting is permitted only when READY or READY WITH WARNINGS, and the agent sees exactly which gates block it and why.

4.9 EXPLANATION GENERATION

Every requirement and issue stores a structured explanation: rule identifier and version, template, facts with their sources (document, page, quote) and, for consistency issues, δ and the tolerance. The text is rendered deterministically, so it always exists and matches the decision; an optional LLM rephrasing is accepted only if every fact and source survives. Explanations thus describe the actual decision procedure [37], not a post-hoc approximation.

4.10 ROLE OF THE LLM AND GUARDRAILS

The LLM is confined to perception tasks, reading documents and wording text, and never makes decisions (Table 4.5). If it is unavailable, values can be entered or confirmed manually and every engine keeps working.

Table 4.5 Use of the LLM and deterministic controls

Task	LLM used	Deterministic control
Field extraction from documents	Yes	JSON Schema validation, quote grounding, rule-based confidence
Mapping unknown labels to canonical fields	Suggests	Restricted to dictionary keys; confirmation for high-materiality fields
Rephrasing explanations	Optional	Fact-preservation check with template fallback
Document type check	No	Keyword signatures; mismatch shown to the agent
Requirement determination	No	Rule engine over facts
Consistency decisions and tolerances	No	Comparators and configured tolerances
Readiness assessment	No	Deterministic gates
Underwriting decisions	No	Human underwriter only

4.11 UNDERWRITER REVIEW WORKFLOW AND AUDIT TRAIL

Table 4.6 defines the underwriter actions; resolution actions require a note.

Table 4.6 Underwriter actions and their effects

Action	Effect	Recorded in audit
Mark issue reviewed	Issue becomes REVIEWED and remains visible	Actor, time, note
Request additional information	Information request with due date; new requirement for the agent; submission INFO REQUESTED	Request text, due date

Action	Effect	Recorded in audit
Accept supporting evidence	Selected evidence item becomes the basis for the field; related conflict resolved	Evidence identifier, note
Resolve conflict	Value to carry forward is selected; conflict resolved	Chosen value, sources, note
Refer submission	Submission REFERRED to a senior underwriter when an issue exceeds authority	Reason, referral target
Move forward	Submission moved forward to quoting; allowed only with no open High issue	Readiness snapshot

Each action writes an AuditEvent with actor, role, action, entity, before and after values, reason and timestamp. Events are append-only and hash-chained,

$$h_k = \text{SHA-256}(h_{k-1} \parallel \text{payload}_k) \quad (4.5)$$

so altering or deleting any past event invalidates every later hash. The History page shows the trail as a timeline.

4.12 EVALUATION PLAN

Evaluation will use 20 to 30 synthetic submissions with four to seven template-generated documents each, seeded with value conflicts of 15–50%, location mismatches, benign name variants, expired certificates, missing documents, undeclared hazardous material and lakh/crore variants, and labelled with ground-truth requirements and issues. Unit tests for every comparator and normalizer, rule-pack tests per scenario and integration tests with a stubbed LLM complement it. Table 4.7 lists Phase II targets, not results.

Table 4.7 Evaluation metrics and targets

Aspect	Metric	Target
Extraction	Field-level precision, recall and F1 against ground truth	$F1 \geq 0.90$ on text-layer PDFs
Grounding	Share of accepted values with a verified quote	100%
Conflict detection	Recall and precision on seeded conflicts	$\text{Recall} \geq 0.95$, $\text{precision} \geq 0.90$
False positives	Conflicts raised on benign variants of names and units	$\leq 5\%$ of benign cases
Requirements	Exact match of the requirement set with expert labels	$\geq 95\%$ of submissions
Explanations	Requirements and issues with a complete trace	100%
Determinism	Identical results on repeated evaluation	100%
Performance	Re-evaluation time for a typical submission	< 2 s
Usability	System Usability Scale with 5–8 participants [46]	Score ≥ 68

CHAPTER 5

PROJECT WORK PLAN

5.1 WORK BREAKDOWN AND TIMELINE

The work has six phases: (0) definition, with problem, solution, novelty and scope frozen; (1) design, in this report; (2–3) a two-day build sprint; (4) evaluation with tolerance tuning; and (5) deployment and the Phase-II report. Phases 0 and 1 are complete (Fig. 5.1).

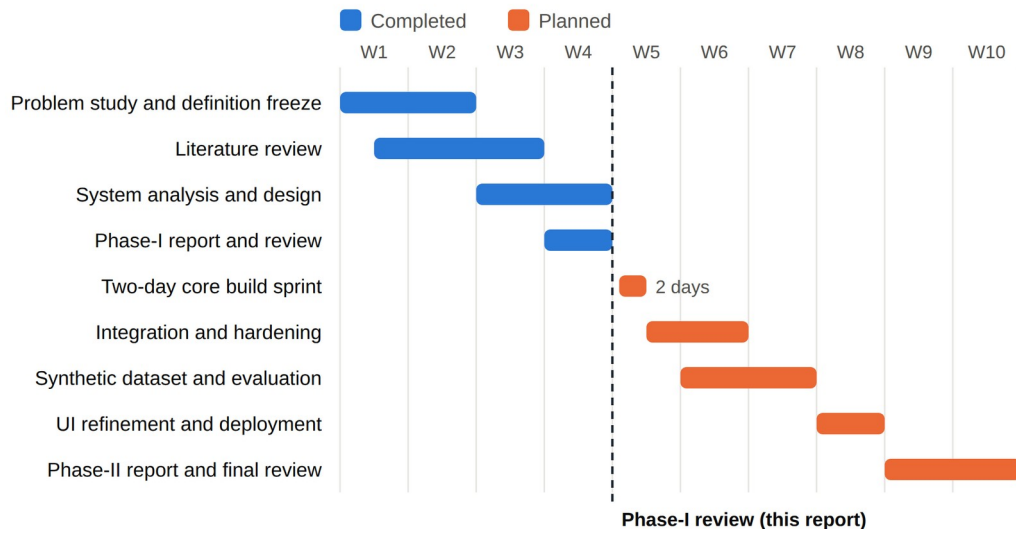


Figure 5.1 Project timeline

5.2 TWO-DAY BUILD SPRINT PLAN

Each sprint session (Table 5.1) ends with a demonstrable exit criterion; the deterministic engines work before extraction is connected, so the core can be shown with manual evidence even if extraction slips.

Table 5.1 Two-day build sprint plan

Session	Tasks	Exit criterion
Day 1, morning	Repository; Vite, React and TypeScript front end; FastAPI; PostgreSQL in containers; authentication, roles and schema	Agent and underwriter can log in
Day 1, afternoon	Submission creation; questionnaire with visibility rules; Requirement Engine with Commercial Property and General Liability rule packs	Running example yields Table 4.2 with explanations
Day 1, evening	Upload, hashing and versioning; text layer and OCR; background processing status	Uploaded PDFs show page text
Day 2, morning	Schema-guided extraction with grounding check; normalization library with unit tests	Application and SOV fields extracted with page and quote
Day 2, afternoon	Consistency Engine, severity and readiness gates; evidence comparison view	₹12M against ₹18M conflict detected and explained

Session	Tasks	Exit criterion
Day 2, evening	Underwriter dashboard, actions, information requests and audit trail; seed data; deployment	Demonstration scenario runs end to end

5.3 SCOPE CONTROL AND THE NOVELTY INVARIANT

The greatest risk is drift: under time pressure the build could become a generic “AI-powered insurance document management system”, recreating existing products and losing the contribution. A feature is therefore accepted only if it strengthens adaptive requirements, cross-document consistency or explainable assurance (Table 5.2).

Table 5.2 MoSCoW prioritization under the novelty invariant

Priority	Features
Must have	Requirement Engine with explanations and evidence-triggered rules; evidence model with provenance; Consistency Engine with tolerances and severity; readiness gates; evidence comparison view; underwriter actions; audit trail
Should have	OCR for scanned pages; LLM label mapping; General Liability rule pack; rule-pack upload; hash-chained audit
Could have	Notifications for information requests; dashboard charts; LLM rephrasing of explanations
Will not have now	Rating and quoting; PolicyCenter integration; fraud scoring; external risk-data APIs; vector databases and embeddings

5.4 RISK ANALYSIS AND MILESTONES

Table 5.3 Risk register

Risk	Likelihood	Impact	Mitigation
LLM extracts wrong or hallucinated values	Medium	High	Quote grounding, rule-based confidence, manual confirmation; engines also work with manual evidence
LLM API cost, rate limits or outage	Medium	Medium	Cache by document hash; small models; manual entry fallback
Tolerances produce false conflicts	Medium	Medium	Per-field configuration; warning band; tuning on the synthetic set
Scope drift towards document management	High	High	Novelty invariant, MoSCoW and the frozen definition
Two-day sprint overruns	High	Medium	Deterministic-first order; Must items only; seeded demonstration data
Privacy concerns	Low	High	Synthetic data only; minimization; masked identifiers

Milestones: M1–M2 definition and design (this report); M3 engines passing the Chapter 4 worked examples; M4 end-to-end prototype; M5 evaluation against Table 4.7; M6 deployment and Phase-II report.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This Phase-I report has defined and designed an Intelligent Adaptive Insurance Submission and Underwriting Assurance System for commercial insurance, oriented to the Guidewire PolicyCenter submission workflow. Checklist-driven intake verifies that documents are present but not that their contents agree or suffice, which causes incomplete submissions, late discovery of inconsistencies and repeated requests for information.

The design treats every piece of information as evidence with provenance, determines requirements from both answers and document content, checks evidence across documents with explicit tolerances, and explains every requirement and issue through the rule and facts that produced it. Decisions remain deterministic and auditable; the LLM only reads documents and words text, and underwriters make every decision. The contribution is deliberately narrow and defensible: not document extraction or adaptive forms, which are established, but an evidence-assurance layer that continuously evaluates the consistency and sufficiency of an evolving submission and explains its additional requirements. The architecture, data model, algorithms, worked examples and plans presented here form a complete basis for implementation in Phase II.

6.2 FUTURE ENHANCEMENTS

- **PolicyCenter integration:** raising underwriting issues in PolicyCenter from the consistency engine through its cloud APIs.
- **More lines of business:** rule packs and field dictionaries for Workers' Compensation, Commercial Auto and Marine.
- **Learning from decisions:** calibrating tolerances and source-authority orders from underwriter resolutions while keeping decisions rule-based.
- **Broader evidence:** optional geocoding and hazard data, multilingual documents such as Tamil and Hindi, and structured ACORD data.
- **Rule-pack verification:** detection of conflicting or unreachable rules and a visual DMN-style rule editor [10].

REFERENCES

- [1] ACORD Corporation, *ACORD 125 Commercial Insurance Application* and *ACORD 140 Property Section*. Pearl River, NY, USA: ACORD. [Online]. Available: <https://www.acord.org>
- [2] Guidewire Software, Inc., “PolicyCenter: Insurance policy administration system,” product overview. [Online]. Available: <https://www.guidewire.com/products/core-products/insurancesuite/policycenter-insurance-policy-administration>
- [3] Guidewire Software, Inc., “Underwriting issues in PolicyCenter,” in *PolicyCenter Cloud API Business Flows and Configuration Guide*, release 202411. [Online]. Available: <https://docs.guidewire.com/cloud/pc/202411/cloudapibf/>
- [4] Guidewire Software, Inc., “Referral reasons,” in *PolicyCenter Cloud API Business Flows and Configuration Guide*, release 202507. [Online]. Available: <https://docs.guidewire.com/cloud/pc/202507/cloudapibf/>
- [5] M. Reilly, “Why underwriters don’t underwrite much,” *Accenture Insurance Blog*, 2021. [Online]. Available: <https://insuranceblog.accenture.com/why-underwriters-dont-underwrite-much>
- [6] M. Eling, D. Nuessle, and J. Staubli, “The impact of artificial intelligence along the insurance value chain and on the insurability of risks,” *The Geneva Papers on Risk and Insurance – Issues and Practice*, vol. 47, no. 2, pp. 205–241, 2022.
- [7] European Insurance and Occupational Pensions Authority (EIOPA), “Artificial intelligence governance principles: Towards ethical and trustworthy artificial intelligence in the European insurance sector,” Report of the Consultative Expert Group on Digital Ethics in Insurance, Frankfurt, Germany, Jun. 2021.
- [8] National Association of Insurance Commissioners (NAIC), “NAIC Model Bulletin: Use of artificial intelligence systems by insurers,” adopted Dec. 4, 2023.
- [9] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of text and layout for document image understanding,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 1192–1200.
- [10] Object Management Group, *Decision Model and Notation (DMN), Version 1.5*, Aug. 2024. [Online]. Available: <https://www.omg.org/spec/DMN/1.5>
- [11] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [12] P. Buneman, S. Khanna, and W.-C. Tan, “Why and where: A characterization of data provenance,” in *Proc. 8th Int. Conf. Database Theory (ICDT)*, LNCS 1973, 2001, pp. 316–330.
- [13] B. G. Buchanan and E. H. Shortliffe, Eds., *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Reading, MA, USA: Addison-Wesley, 1984.
- [14] K. S. Aggour, P. P. Bonissone, W. E. Cheetham, and R. P. Messmer, “Automating the underwriting of insurance applications,” *AI Magazine*, vol. 27, no. 3, pp. 36–50, 2006.
- [15] C. L. Forgy, “Rete: A fast algorithm for the many pattern/many object pattern match problem,” *Artificial Intelligence*, vol. 19, no. 1, pp. 17–37, 1982.
- [16] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, “LayoutLMv3: Pre-training for document AI with unified text and image masking,” in *Proc. 30th ACM Int. Conf. Multimedia*, 2022, pp. 4083–4091.
- [17] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, “OCR-free document understanding transformer,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 498–517.
- [18] G. Jaume, H. K. Ekenel, and J.-P. Thiran, “FUNSD: A dataset for form understanding in noisy scanned documents,” in *Proc. Int. Conf. Document Analysis and Recognition Workshops (ICDARW)*, 2019, pp. 1–6.
- [19] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [20] D. Xu, W. Chen, W. Peng, C. Zhang, T. Xu, X. Zhao, X. Wu, Y. Zheng, Y. Wang, and E. Chen, “Large language models for generative information extraction: A survey,” *Frontiers of Computer Science*, vol. 18, no. 6, Art. no. 186357, 2024.

- [21] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, Art. no. 248, pp. 1–38, 2023.
- [22] I. P. Fellegi and A. B. Sunter, “A theory for record linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.
- [23] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Berlin, Germany: Springer, 2012.
- [24] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [25] W. E. Winkler, “String comparator metrics and enhanced decision rules in the Fellegi–Sunter model of record linkage,” in *Proc. Section on Survey Research Methods, American Statistical Association*, 1990, pp. 354–359.
- [26] J. Bleiholder and F. Naumann, “Data fusion,” *ACM Computing Surveys*, vol. 41, no. 1, Art. no. 1, pp. 1–41, 2009.
- [27] Y. Li, J. Gao, C. Meng, Q. Li, L. Su, B. Zhao, W. Fan, and J. Han, “A survey on truth discovery,” *ACM SIGKDD Explorations Newsletter*, vol. 17, no. 2, pp. 1–16, 2016.
- [28] X. L. Dong, L. Berti-Equille, and D. Srivastava, “Integrating conflicting data: The role of source dependence,” *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 550–561, 2009.
- [29] R. Y. Wang and D. M. Strong, “Beyond accuracy: What data quality means to data consumers,” *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.
- [30] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, “Methodologies for data quality assessment and improvement,” *ACM Computing Surveys*, vol. 41, no. 3, Art. no. 16, pp. 1–52, 2009.
- [31] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: A large-scale dataset for fact extraction and VERification,” in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2018, pp. 809–819.
- [32] W. R. Swartout, “XPLAIN: A system for creating and explaining expert consulting programs,” *Artificial Intelligence*, vol. 21, no. 3, pp. 285–325, 1983.
- [33] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [34] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.
- [35] A. Barredo Arrieta *et al.*, “Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [36] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” arXiv:1702.08608, 2017.
- [37] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.
- [38] Insurance Regulatory and Development Authority of India (IRDAI), “IRDAI Information and Cyber Security Guidelines, 2023,” Hyderabad, India, Apr. 2023.
- [39] Government of India, *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023). New Delhi, India: Ministry of Law and Justice, 2023.
- [40] Meta Platforms, Inc., “React documentation.” [Online]. Available: <https://react.dev>
- [41] S. Ramírez, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com>
- [42] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015.
- [43] The PostgreSQL Global Development Group, “PostgreSQL documentation.” [Online]. Available: <https://www.postgresql.org/docs/>
- [44] OWASP Foundation, “OWASP Top 10: 2021.” [Online]. Available: <https://owasp.org/Top10/>
- [45] JSON Schema, “JSON Schema: A media type for describing JSON documents,” Draft 2020-12. [Online]. Available: <https://json-schema.org/draft/2020-12>
- [46] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.